

Exercício 4.2 - Modelo de estados

a) Modelo de 2 estados

A implementação com um modelo de 2 estados é uma versão simplificada e que funciona como preparação para o 1º trabalho.

Considere um modelo de 2 estados: apenas READY e RUN

Neste modelo simplificado não existe estado "new" nem "exit" nem "Blocked".

Quando os processos estão em "Ready", estão numa fila de espera do tipo FIFO (first in first out), a este tipo de escalonamento dá-se o nome de "FCFS - First Come First Served".

Os processos são definidos por um conjunto de pseudo-instruções-máquina representadas por números inteiros positivos, que representam:

- o código de uma instrução que executa durante **1 instante** o processo fica no CPU para essa instrução (estado RUNNING) até ser interrompido

Por exemplo 2,3,5,0,0... significa que o processo (quando chegar a sua vez)..

- Corre a instrução com o código 2 por um instante

- Após a execução de uma instrução (que dura 1 instante), considera-se que o processo é interrompido, o que faz com o processo volte ao fim da fila FIFO.

- Após correr a instrução com o código 3 por um instante, o processo interrompe novamente, depois quando chega novamente a sua vez, corre a instrução 5 por um instante e interrompe novamente;

- Finalmente, é executada a instrução HALT, que é representada por um 0 (zero). Esta instrução executa durante um instante. O processo termina imediatamente após esse instante.

Nota: Pode assumir que se um zero (HALT) é encontrado, as instruções seguintes também serão zero.

- No total, o processo corre $1+1+1+1 = 4$ instantes no CPU (RUNNING);

Considere os processos definidos pelos seguintes 3 pseudo-programas que iniciam todos no instante zero e entram no sistema conforme a seguinte ordem (ou seja, o P1 entra primeiro na fila, depois o P2 e em último o P3):

```
int programas[5][3] = { {3, 4, 2},
```

```
{1, 2, 1},  
{2, 4, 5},  
{0, 0, 10},  
{0, 0, 0} };
```

Implemente um simulador dos processos no sistema.

OUTPUT: o programa deve ter como output, o estado de cada processo em cada instante.
Por exemplo:

instante	proc1	proc2	proc3
1	RUN	READY	READY
2	READY	RUN	READY
3	READY	READY	RUN
4	READY	RUN	READY
...etc...			

Orientações para a implementação - Comece por:

- 1) definir que dados deve ter o processo.
- 2) implementar uma fila genérica para gerir os processos na fila de READY
- 3) implementar as funções que movem os processos entre a fila de READY e RUN

b) Instruções de I/O

Em vez de cada processo interromper após a execução de cada instrução, agora considere que existe um conjunto de instruções de I/O que interrompem o processo. O processo apenas interrompe após correr uma instrução de I/O.

O código destas instruções é dado por um número inteiro negativo.

Por exemplo 2,3,-5,1,-3,0... significa que o processo (quando chegar a sua vez) ...

--- Corre a instrução com o código 2 por um instante, a instrução com o código 3 por um instante e corre a instrução de I/O com o código -5 por um instante e interrompe, vai para o fim da fila.

– Quando é novamente a sua vez, o processo corre a instrução 1 por um instante, depois corre a instrução -3 por um instante e interrompe.

– Quando volta, corre a instrução 0 (HALT) por um instante e termina.

– Resumindo, o processo corre 3 instantes seguidos no CPU (RUNNING), interrompe. Quando volta a correr, corre 2 instantes e interrompe. Quando volta, corre 1 instante e termina.

Considere os processos definidos pelos seguintes 3 pseudo-programas que iniciam todos no instante zero e entram no sistema conforme a seguinte ordem (ou seja, o P1 entra primeiro na fila, depois o P2 e em último o P3):

```
int programas[5][3] = { {3, 4, 2},  
                        {1, -2, 1},  
                        {-5, 4, 5},  
                        {1, 0, 10},  
                        {0, 0, 0} };
```

Implemente um simulador dos processos no sistema.

OUTPUT: o programa deve ter como output, o estado de cada processo em cada instante.

Por exemplo:

instante	proc1	proc2	proc3
1	RUN	READY	READY
2	RUN	READY	READY
3	RUN	READY	READY
4	READY	RUN	READY
...etc...			